



SymfonyOnline
JUNE 2026

JUNE 11 – 12, 2026



ENGLISH TALKS

#SYMFONYONLINE

Jérôme Tamarelle

**“PROTECT YOUR DATA WITH
QUERYABLE ENCRYPTION”**

JOIN US!



X @GromNaN





**YOUR DATA HAVE BEEN
COMPROMISED!**



PII – Personally Identifiable Information



1. Direct Identification

- **First Name / Last Name** (Jérôme Tamarelle)
- **Social Security number and passport number**
- **Email address** (jerome@tamarelle.net).
- **Social media** (@GromNaN)

2. Indirect Identification

- **Technical identifiers:** IP address, credit card
- **Contextual data:** Geolocation, license plate number.
- **Specificities :** "PHP developer, living in Rouen and working for MongoDB"



INDUSTRY-FIRST

MongoDB Queryable Encryption

A unique **in-use encryption** technology protects sensitive data during storage, processing, and access throughout its entire lifecycle. It enables applications to encrypt sensitive data on the **client side**, store it securely in the MongoDB database, and perform **complex queries directly on the encrypted data**.

Encryption: hides the data; can be reversed through decryption

Algorithmes	Caractéristiques	Date	Crypto-Safe
Caesar	Single-character substitution	c. 50 B.C.	No
Vigenère	Poly-alphabetic substitution	1553	No
Enigma Machine	Rotor-based electromechanics	1918	No
DES (Data Encryption Standard)	Block cipher (56-bit)	1977	Outdated
RSA (Rivest-Shamir-Adleman)	Asymmetric (Public/private key)	1977	Yes
AES-256-GCM (Advanced Encryption Standard)	Symmetric (Block - SSAE) Post-quantum	2001 (AES) 2007 (GCM)	Yes



Hash: a unique, fixed, and irreversible digital signature of a piece of data.

Algorithms	Main usage	Date	"Crypto-Safe" ?
CRC32	Error Detection (Network)	1975	No
MD5	File integrity	1991	No (Broken)
SHA-1	Digital signature	1995	No (Obsolete)
Bcrypt	Password hashing	1999	No (Unsalted)
SHA-256	Standard Security & Bitcoin	2001	Yes
xxHash (XXH3)	Performance / Big Data	2012 / 2019	No
Argon2	Password hashing	2015	Yes

A CRYPTO NERD'S
IMAGINATION:

HIS LAPTOP'S ENCRYPTED.
LET'S BUILD A MILLION-DOLLAR
CLUSTER TO CRACK IT.

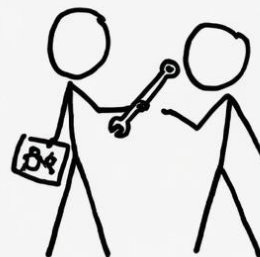
BLAST! OUR
EVIL PLAN
IS FOILED!

NO GOOD! IT'S
4096-BIT RSA!

WHAT WOULD
ACTUALLY HAPPEN:

HIS LAPTOP'S ENCRYPTED.
DRUG HIM AND HIT HIM WITH
THIS \$5 WRENCH UNTIL
HE TELLS US THE PASSWORD.

GOT IT.



Encryption

```
function encrypt(string $plaintext, string $dek): string
{
    $iv = // Initialisation Vector

    $ciphertext = openssl_encrypt($plaintext, 'aes-256-gcm', $dek, OPENSSL_RAW_DATA, $iv);
    return $iv . $ciphertext;
}
```


Random encryption

The result of the encryption operation changes randomly

```
function encrypt(string $plaintext, string $dek): string
{
    $iv = random_bytes(16);

    $ciphertext = openssl_encrypt($plaintext, 'aes-256-gcm', $dek, OPENSSL_RAW_DATA, $iv);
    return $iv . $ciphertext;
}
```

Deterministic encryption

The result of the encryption operation is always the same for a given value

```
function encrypt(string $plaintext, string $dek): string
{
    $hash = hash('sha512', $plaintext, true);
    $iv = substr($hash, 0, 16);

    $ciphertext = openssl_encrypt($plaintext, 'aes-256-gcm', $dek, OPENSSL_RAW_DATA, $iv);
    return $iv . $ciphertext;
}
```

The hash value can be compared with other databases!

Deterministic encryption

The result of the encryption operation is always the same for a given value

```
function encrypt(string $plaintext, string $dek): string
{
    $hmac = hash_hmac('sha512', $plaintext, $dek);
    $iv = substr($hmac, 0, 16);

    $ciphertext = openssl_encrypt($plaintext, 'aes-256-gcm', $dek, OPENSSL_RAW_DATA, $iv);

    return $iv . $ciphertext;
}
```

(MongoDB uses a different derived key for hashing and encryption)

Decryption

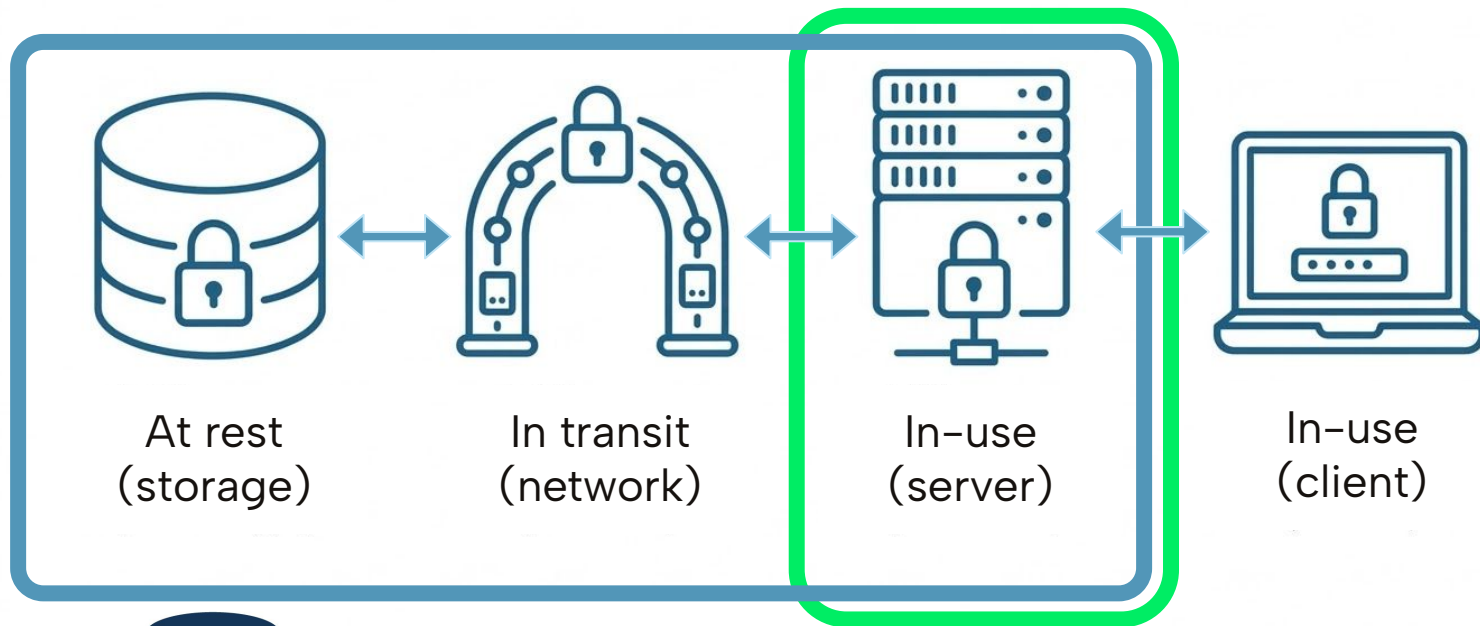
The result of the encryption operation is always the same for a given value

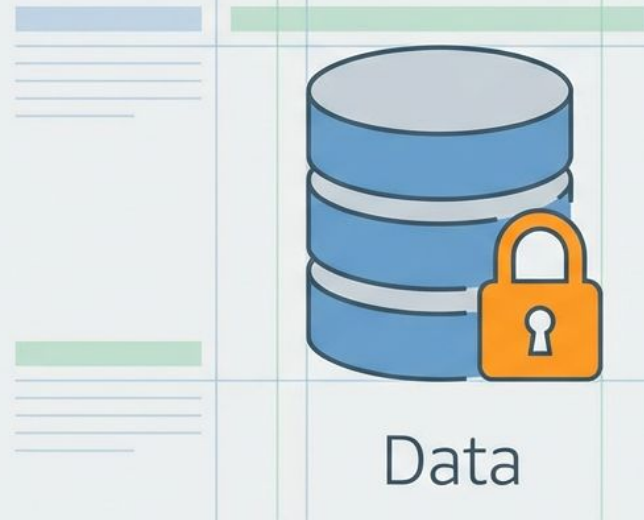
```
public function decrypt(string $payload, string $dek): string
{
    // Split IV & ciphertext.
    $iv = substr($payload, 0, 16);
    $ciphertext = substr($payload, 16);

    $plaintext = openssl_decrypt($ciphertext, 'aes-256-gcm', $dek, \OPENSSL_RAW_DATA, $iv);

    if ($plaintext === false) {
        throw new RuntimeException('Decryption failed: ' . openssl_error_string());
    }

    return $plaintext;
}
```

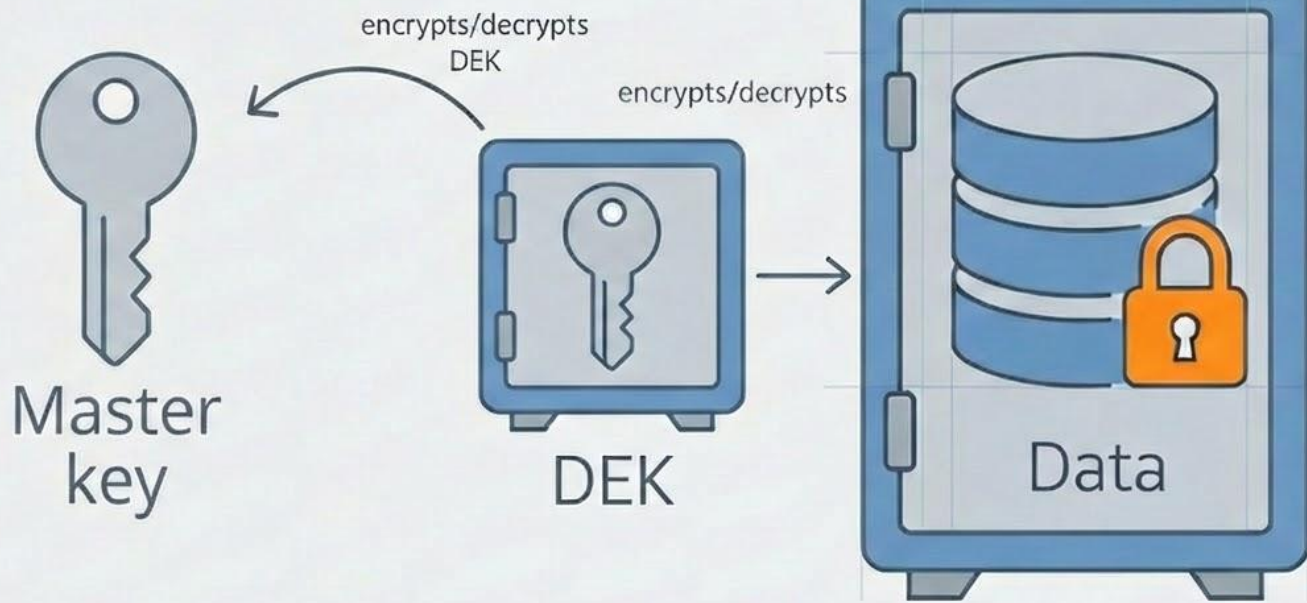




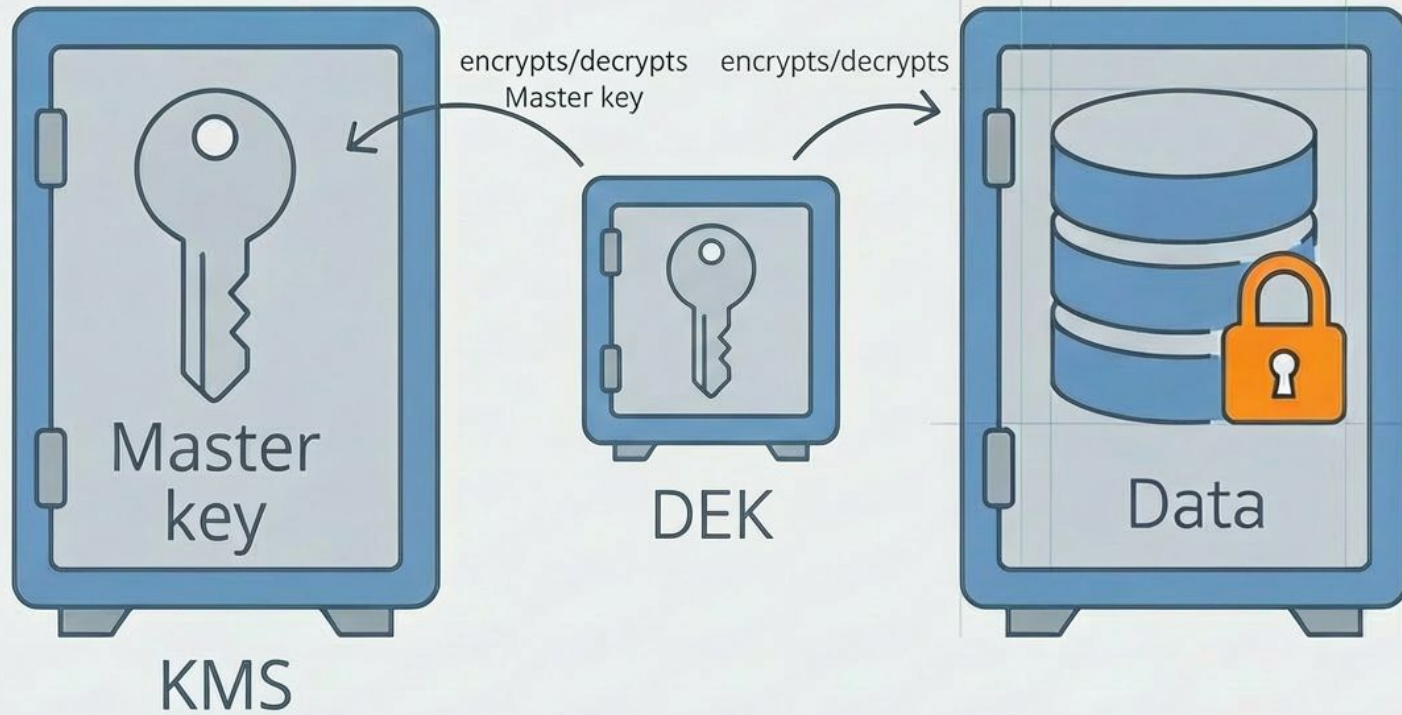
1. Sensitive data requires protection



1. Sensitive data requires protection
2. This data is encrypted using a DEK (Data Encryption Key)



1. Sensitive data requires protection
2. This data is encrypted using a DEK (Data Encryption Key)
3. The DEK is secured by a Master Key



1. Sensitive data requires protection
2. This data is encrypted using a DEK (Data Encryption Key)
3. The DEK is secured by a Master Key
4. The KMS (Key Management System) protects the Master Key



Choose a Key Management System (KMS)



`tmpfs:///secrets/master.key`



[Kms] Add a new component for Key Management Systems #64052

Open

Spomky wants to merge 1 commit into symfony:8.1 from Spomky:kms

Conversation 47

Commits 1

Checks 11

Files changed 159

+10,628 -60

Spomky commented last month • edited

Member

Q	A
Branch?	8.1 (will retarget once 8.2 is cut)
Bug fix?	no
New feature?	yes
Deprecations?	no
Issues	-
License	MIT
Docs PR	symfony/symfony-docs#22280

This PR introduces a new `Symfony\Component\Kms` component that abstracts Key Management Systems behind a unified API. Initial discussion was kicked off by [@GromNaN](#) around offering something better than `secret:encrypt` by leveraging managed KMS backends.

Component

Reviewers

GromNaN

stof

Copilot

Request

Still in progress? Convert to draft

Assignees

No one—assign yourself

Labels

Feature

Status: Needs Review

Projects

None yet

Milestone

8.2

Field-level encryption in the database

```
#[ORM\Entity]
class User
{
    #[ORM\Id]
    #[ORM\Column]
    public ?int $id = null;

    // Email unique
    #[ORM\Column(length: 512, type: Types::STRING, unique: true)]
    public string $email;

    #[ORM\Column(length: 512, type: Types::STRING)]
    public string $firstName;

    #[ORM\Column(length: 512, type: Types::STRING)]
    public string $lastName;
}
```

Field-level encryption in the database

```
#[ORM\Entity]
class User
{
    #[ORM\Id]
    #[ORM\Column]
    public ?int $id = null;

    // Deterministic encryption of the email enables unique indexing
    #[ORM\Column(length: 512, type: Types::STRING, unique: true)]
    public string $email;

    // Random encryption prevents correlating identical names
    #[ORM\Column(type: Types::BINARY)]
    public string $firstName;

    #[ORM\Column(type: Types::BINARY)]
    public string $lastName;
}
```

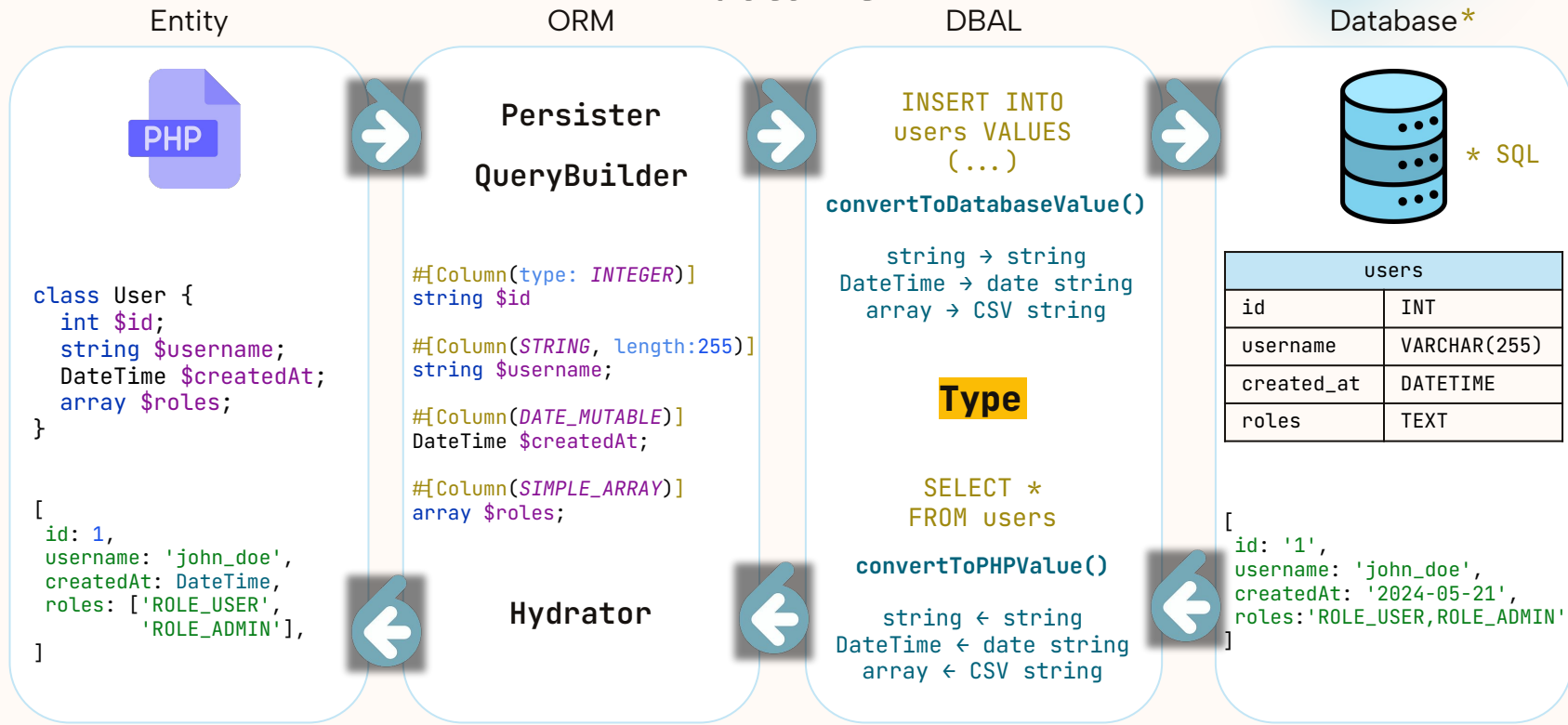
Field-level encryption in the database

```
$user = new User();  
$user→email = $encryptor→encryptDeterministic('fabien@symfony.com', $dek);  
$user→firstName = $encryptor→encryptRandom('Fabien', $dek);  
$user→lastName = $encryptor→encryptRandom('Potencier', $dek);
```

```
$encryptor→decrypt($user→email, $dek); // fabien@symfony.com  
$encryptor→decrypt($user→firstName, $dek); // Fabien  
$encryptor→decrypt($user→email, $dek); // Potencier
```


Field-based encryption with the ORM





Creating a custom DBAL Type

```
final readonly class EncryptedType extends \Doctrine\DBAL\Types\Type
{
    public function convertToDatabaseValue(mixed $value, AbstractPlatform $platform): mixed
    {
        // ...
    }

    public function convertToPHPValue(mixed $value, AbstractPlatform $platform): mixed
    {
        // ...
    }
}
```

Creating a custom DBAL Type

```
public function convertToDatabaseValue(mixed $value, AbstractPlatform $platform): mixed
{
    $plaintext = $this->parentType->convertToDatabaseValue($value, $platform);

    if ($plaintext === null) {
        return null;
    }

    $cypherblob = $this->encryptor->encryptRandom($this->dekId, $plaintext);

    return $cypherblob;
}
```

Creating a custom DBAL Type

```
public function convertToPHPValue(mixed $value, AbstractPlatform $platform): mixed
{
    if (!$value) {
        return null;
    }

    $plaintext = $this->encryptor->decrypt($this->dekId, $value);

    return $this->parentType->convertToPHPValue($plaintext, $platform);
}
```

Creating a custom DBAL Type

```
public function getBindingType(): ParameterType
{
    return ParameterType::BINARY;
}
```

```
public function getSQLDeclaration(array $column, AbstractPlatform $platform): string
{
    return $platform->getBinaryTypeDeclarationSQL($column);
}
```

Creating a custom DBAL Type

```
use Doctrine\DBAL\Types\Type;
use App\Encryption\DekEncryptionService;

final readonly class EncryptedType extends Type
{
    public function __construct(
        private Type $parentType,
        private Encryptor $encryptor,
        private string $dekId,
    ) {}
}
```

Remove final constructor for Type #6705

Code ▾

Merged

greg0ire merged 4 commits into doctrine:4.3.x from ruudk:ruudk/2025/1/remove-final-type-constructor on Feb 20, 2025

Conversation 31 Commits 4 Checks 66 Files changed 4 +89 -12



ruudk commented on Jan 13, 2025 • edited ▾

Contributor ⋮

Q	A
Type	improvement
Fixed issues	#6704

Summary

This allows people to define a constructor when creating custom Doctrine\DBAL\Types\Type implementations.

They can be registered by passing an instance to Type::addType .

When a class string is passed, and the Type cannot be instantiated, an helpful error message will be thrown.



ruudk mentioned this pull request on Jan 13, 2025

Documentation about Custom Mapping Types with custom constructor is invalid #6704

Closed

Reviewers

- greg0ire ✓
- SenseException ✓
- derrabus ●
- +1 more reviewer ^
- GromNaN ✓

Assignees

No one assigned

Labels

Improvement

Projects

None yet

Milestone

4.3.0

Register a custom DBAL Type

```
$type = new EncryptedType(  
    Type::getTypeRegistry()→get('string'),  
    $container→get(Encryptor::class),  
    'my-dek-id',  
);  
  
Type::getTypeRegistry()→register('user_name_encrypted', $type);
```

Add #[AsDatabaseType] attribute to register DBAL types as Symfony services #2221

[Merge conflicts](#)[Code](#)[Open](#)

GromNaN wants to merge 4 commits into doctrine:3.3.x from GromNaN:feat/dbal-type-as-service

[Conversation](#) 4[Commits](#) 4[Checks](#) 13[Files changed](#) 9

+692 -2



GromNaN commented on Apr 3 • edited

Member

Introduces the #[AsDatabaseType] PHP attribute and a DatabaseTypePass compiler pass to register Doctrine DBAL types as Symfony services with a per-connection TypeRegistry, without touching the global static type registry.

Example

```
use Doctrine\Bundle\DoctrineBundle\Attribute\AsDatabaseType;
use Doctrine\DBAL\Platforms\AbstractPlatform;
use Doctrine\DBAL\Types\Type;

#[AsDatabaseType(name: MoneyType::NAME)]
final class MoneyType extends Type
{
    public const string NAME = 'money';

    public function getSQLDeclaration(array $column, AbstractPlatform $platform): string
    {
        return $platform->getDecimalTypeDeclarationSQL($column);
    }
}
```

```
#[ORM\Column(type: MoneyType::NAME)]
private Money $price;
```

Dependencies

This PR depends on changes in DBAL and ORM to support per-connection TypeRegistry :

- [Add TypeRegistry to Configuration and use it in all internal type resolution](#) dbal#7342
- [Replace static Type::* calls with instance-based TypeRegistry lookups](#) orm#12421

Reviewers

stof

Copilot

[Request](#)

At least 1 approving review is required to merge this pull request.

Still in progress? Convert to draft

Assignees

No one—assign yourself

Labels

None yet

Projects

None yet

Milestone

No milestone

Development

Successfully merging this pull request may close these issues.

None yet

Notifications

[Customize](#)[Unsubscribe](#)

Using a custom DBAL Type

```
#[ORM\Entity]

class User
{
    #[ORM\Id, ORM\GeneratedValue, ORM\Column]
    public ?int $id = null;

    #[ORM\Column(type: 'user_email_encrypted', unique: true)]
    public string $email;

    #[ORM\Column(type: 'user_name_encrypted')]
    public string $name;
}
```

Types are used where fields are identified

- ✓ When hydrating data from the database into entity classes
- ✓ When sending data from entities to the database
- ✓ When querying using the query builder
- ✗ DQL or SQL queries
- ✗ Raw database results

Searching on encrypted data

```
#[ORM\Entity]
class User
{
    // Email encrypted with a random IV
    #[ORM\Column(type: Types::BINARY)]
    public string $email;
```

Searching on encrypted data

```
#[ORM\Entity]
class User
{
    // Email encrypted with a random IV
    #[ORM\Column(type: Types::BINARY)]
    public string $email;

    #[ORM\Column(type: Types::STRING, length: 64)]
    public string $emailHash;

    #[ORM\Column(type: Types::STRING, length: 64)]
    public string $emailDomainHash;
}
```

Searching on encrypted data

```
#[AsDoctrineListener(event: \Doctrine\ORM\Events::prePersist)]
#[AsDoctrineListener(event: \Doctrine\ORM\Events::preUpdate)]

final class UserEmailHashListener
{
    public function prePersist(PrePersistEventArgs $args): void
    {
        $this→applyHashes($args→getObject());
    }

    public function preUpdate(PreUpdateEventArgs $args): void
    {
        $this→applyHashes($args→getObject());

        $em = $args→getObjectManager();
        $uow = $em→getUnitOfWork();
        $metadata = $em→getClassMetadata(UserEmailEncrypted::class);
        $uow→recomputeSingleEntityChangeSet($metadata, $args→getObject());
    }
}
```

Searching on encrypted data

```
final class UserEmailHashListener
{
    public function __construct(private readonly string $hmacKey) {}

    private function applyHashes(UserEmailEncrypted $entity): void
    {
        $normalizedEmail = strtolower(trim($entity->email));
        $entity->emailHash = hash_hmac('sha256', $normalizedEmail, $this->hmacKey);

        $domain = explode('@', $normalizedEmail, 2)[1] ?? '';
        $entity->emailDomainHash = hash_hmac('sha256', $domain, $this->hmacKey);
    }
}
```


Searching on encrypted data

```
final class UserRepository extends ServiceEntityRepository
{
    public function findByEmailDomain(string $domain): array
    {
        $normalizedDomain = strtolower(trim($domain));
        $domainHash = hash_hmac('sha256', $normalizedDomain, $this->hmacKey);

        return $this->createQueryBuilder('u')
            ->andWhere('u.emailDomainHash = :domainHash')
            ->setParameter('domainHash', $domainHash)
            ->getQuery()
            ->getResult();
    }
}
```

Search tags

```
#[ORM\Entity]
class User
{
    #[ORM\Column(type: 'user_email_encrypted')]
    public string $email;

    /** @var list<string> List of hashed search tags */
    #[ORM\Column(type: Types::SIMPLE_ARRAY)]
    public array $searchTags;
}
```



<https://jerome.tamarelle.net/slides/range-encryption-demo.html>

ÉTAPE 1

ÉTAPE 2

ÉTAPE 3

ÉTAPE 4

ÉTAPE 5

ÉTAPE 6

ÉTAPE 7

ÉTAPE 8

Résultat

L'item est trouvé via [13-14], mais plus l'intervalle est étendu, plus la requête est coûteuse

1. Sans arbre - 4/4

Valeur 13

Requête [9, 22]

N0 niveau absent dans cette variante

N1 niveau absent dans cette variante

N2

0

5

10

15

20

25

Segment (inactif)

Segment stocké (valeur 13)

Segment recherché (requête [9-22])

Segment commun (intersection)

Niveau absent dans la variante

Segment commun [13-14]

Coût requête 13 tags

PostgreSQL — résultat

Chiffrement désactivé

id	sensitive_number	tags
----	------------------	------

1	13	{"N2: [13-14]"}
---	----	-----------------

→ La ligne est retenue car le tableau `tags` intersecte

→ le tableau des segments demandés par la requête.

L'item est bien retrouvé, ici grâce au segment exact [13-14].

La démonstration marche, mais elle montre aussi la limite du modèle sans arbre : la requête est **verbeuse** et envoie beaucoup de tags.

On peut maintenant introduire l'arbre pour conserver le même résultat, avec une requête *beaucoup plus compacte*.



SymfonyOnline
JUNE 2026

★ martin-georgiev/postgresql-for-doctrine

↓ `composer require martin-georgiev/postgresql-for-doctrine`

Extends Doctrine with native PostgreSQL support for arrays, JSONB, ranges, PostGIS geometries, text search, ltree, uuid, and 100+ PostgreSQL-specific functions.

Maintainers



Package info

[github.com/martin-georgiev/postgres...](https://github.com/martin-georgiev/postgresql-for-doctrine)
`pkg:composer/martin-georgiev/postgr...`

Fund package maintenance!

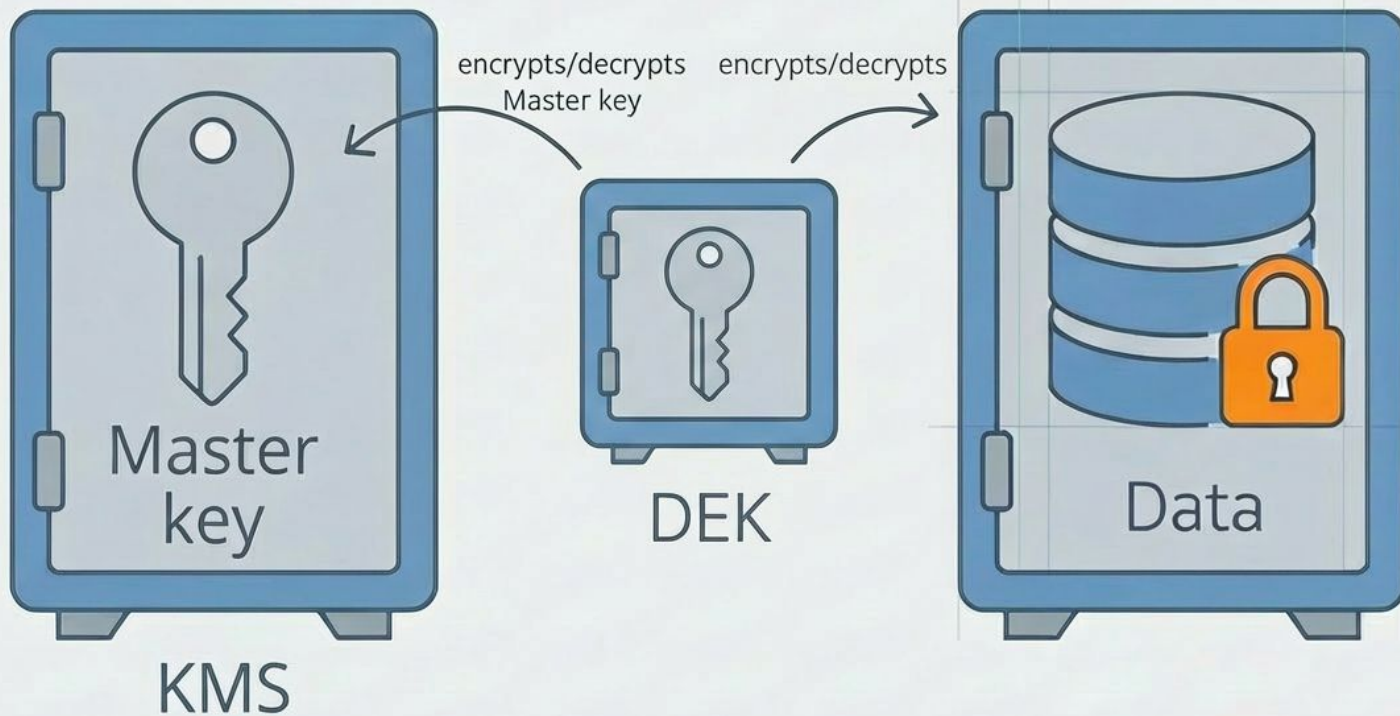
 [martin-georgiev](#)
 [github.com/sponsors/martin-georg...](https://github.com/sponsors/martin-georgiev)

Statistics

Installs:	5 261 907
Dependents:	5
Suggesters:	1
Stars:	447
Open Issues:	10



Key rotation



Key rotation = re-encrypt using the new key

Key to be replaced	Encrypted information	Volume	Duration	Frequency
Master Key	Data Encryption Key	Small as your schema	Fast	6 to 24 months
Data Encryption Key	Business data	Big as your DB	Very long	When needed

Key rotation = re-encrypt using the new key

Encrypted data is always stored along with the identifier for its encryption key

- DEK :
 `id: <id> ,`
 `masterKey: {provider: "aws", key: "<arn>", ...},`
 `keyMaterial: <binary>`
- Encrypted Data : `Binary(<dekId>.<iv>.<cyphertext>)`

#[Doctrine\ODM\MongoDB\Mapping\Attribute\Encrypt]

Doctrine MongoDB ODM

Native support for field-level encryption, queryable

```
use Doctrine\ODM\MongoDB\Mapping\Attribute as ODM;
use Doctrine\ODM\MongoDB\Mapping\EncryptQuery;

#[ODM\Document]
class User
{
    #[ODM\Id]
    public ?int $id = null;

    #[ODM\Field, ODM\Encrypt]
    public string $name;

    #[ODM\Field, ODM\Encrypt(queryType: EncryptQuery::Equality)]
    public string $email;
```

Doctrine MongoDB ODM

Native support for field-level encryption, queryable

```
#[ODM\Field]
#[ODM\Encrypt(
    queryType: EncryptQuery::Range,
    min: new \DateTimeImmutable('1900-01-01'),
    max: new \DateTimeImmutable('2050-01-01'),
)]
public \DateTimeImmutable $birthday;
```

Why choose MongoDB?



- Cross-driver implementation (language- and framework-independent)
- Storage optimized for storage and search performance
- Enhanced security with server-side schema validation
- Editor support
- Other features: document model, Lucene index, vectors, auto-embedding, etc.

What's new in MongoDB 8.2?



ALREADY SUPPORTED

Equality

Retrieve a record for a customer with the exact encrypted email address “alex@example.com”

ALREADY SUPPORTED

Range

Find all salaries ranging from €50,000 to €80,000

NEW

Prefix

Find all customers whose last name starts with “Mac”

NEW

Suffix

Match the last four digits of a numeric Social Security number, such as “-1234”

NEW

Substring

Search for a keyword such as “engineer” in a job title or a numerical description in the free-text field

Why choose Doctrine + Postgres?



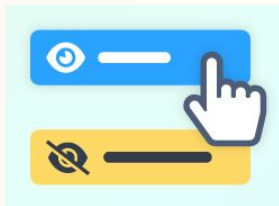
- Fully open-source and customizable implementation
- Gradual addition of column-level encryption to an existing application
- If this is the technology the teams have mastered

Encryption isn't everything: data leaks through user interfaces

Restrict access to
what is strictly
necessary



Click-to-reveal



Rate Limiter



Monitoring and alerts
for abuse





SymfonyOnline
JUNE 2026

JUNE 11 – 12, 2026



ENGLISH TALKS

#SYMFONYONLINE

Jérôme Tamarelle

**“PROTECT YOUR DATA WITH
QUERYABLE ENCRYPTION”**

JOIN US!



X @GromNaN

